

Exp: 04

Write the Python Program for Crypt-Arithmetic Problem.

Aim:

To write a python program for Crypt-Arithmetic problem.

Objective:

To understand constraint satisfaction problems (CSP) by assigning unique digits to letters such that the resulting arithmetic equation holds true.

Algorithm:

Step 1: Write down the arithmetic problem in full, including any carry-overs.

Step 2: Replace each letter with a unique digit from 0 to 9.

Step 3: Generate all possible combinations of digits for each letter, subject to any constraints from the carry-overs and the rules of arithmetic.

Step 4: For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.

Step 5: If a solution is found, output the values of the letters and the arithmetic problem, and exit the algorithm.

Step 6: If no solution is found, output a message indicating that no solution exists.

Program:

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
digits = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

for p in permutations(digits, 8):
    S, E, N, D, M, O, R, Y = p

    if S == 0 or M == 0:
        continue

    SEND = 1000*S + 100*E + 10*N + D
```

$$\text{MORE} = 1000 * M + 100 * O + 10 * R + E$$

$$\text{MONEY} = 10000 * M + 1000 * O + 100 * N + 10 * E + Y$$

```
if SEND + MORE == MONEY:
    print("SEND =", SEND)
    print("MORE =", MORE)
    print("MONEY =", MONEY)
    break
```

Output:

SEND = 9567

MORE = 1085

MONEY = 10652

After defining the `solve_cryptarithmic` function, the code provides an example usage using a

cryptarithmic puzzle: "SEND + MORE = MONEY." It calls the `solve_cryptarithmic`

function with the puzzle string and stores the result in the mapping variable.

If a solution is found (i.e., mapping is not None), it prints the mapping dictionary and the puzzle

string with the letters replaced by their corresponding digits using the `translate` method.

If no solution is found, it prints "No solution found."

Result:

The output has verified.